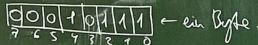


Abschnitt 3: Programmieren

Der Arbeitsspeicher: Array/Line von Speicherzellen einer gegebenen Bitgröße

0 1 ... 9 10 11 12 13 14 15



Objekte im Speicher

haben die Adresse und die Größe (einfache Objekte)

$\&a \leftarrow$ Adresse von a
 $\text{sizeof}(a) \leftarrow$ Größe in Bytes. } C/C++

①

Ein Programm nutzt Speicher, der in die folgenden Segmente eingeteilt wird:

- Code segment



- Data Segment (die globalen Variablen).

- Programmstack (für die lokalen Variablen für die Organisation der Aufrufe von Funktionen).

- Dynamischer Speicher (Reservierung der Speicherblöcke variabler Größe, z.B. für Arrays, deren Größe variabel ist.)

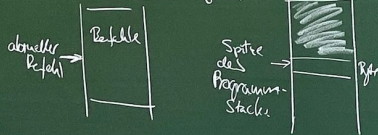
Neben dem Speicher hat man noch einen Prozessor mit Registern

Insbesondere

- ein Register mit der Adresse des aktuellen Befehls

- ein Register mit der Adresse des Programm-Stack-Spitze

- Weitere Register

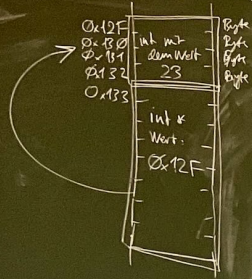


Grundbefehle:

- Arithmetik
- Daten Kopieren (Speicher, Register)
- Kontrollfluss fehler

②

Eine Adresse einer Speicher-Zelle kann im Speicher aufgeschrieben werden. Das entsprechende Objekt im Speicher nennt man einen Zeiger.



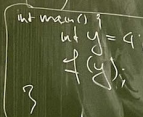
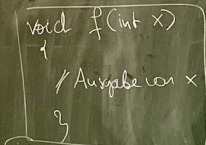
$\text{sizeof(int)} == 4$

$\text{sizeof(int*)} == 8$

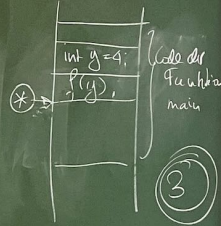
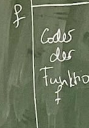
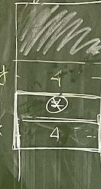
In C/C++:

xp ist Inhalt unter der Adresse p , wenn p eine Zeigervariable ist.

Wie ist die Ausführung von Programmfunktionen angeordnet? Programmfunktionen nennt man auch Prozeduren.



Programmstack



Wenn wir durch den Aufruf $g(y)$ das Objekt y ändern wollen, wie soll man g umsetzen?

`void g(int* p) {`

`(*p)++;`

`}`

Übergabe durch Zeiger

`int main {`

`int y=4;`

`g(&y);` // y wird um 1 erhöht

`}`

Alternative:

`void h(int& x) {`

`x++;`

`}`

Übergabe durch Referenz

`int main {`

`int y=4;`

`h(y);` // y wird um 1 erhöht

`}`

Bei Funktionen mit Rückgabewerten landet die Rückgabe auf dem Programmstack

④

Wenn wir durch den Aufruf $g(y)$ das Objekt y ändern wollen, wie soll man g umsetzen?

void $g(\text{int}^* p)$

$(*p)++$;

}

Übergabe
durch Zeiger

int main {
int $y = 4$;

$g(y)$; // y wird um 1 erhöht

}

Alternative:

void $h(\text{int} \& x)$

$x++$;

}

Übergabe
durch
Referenz.

int main {

int $y = 4$;

$h(y)$;

// y wird um 1 erhöht

}

Bei Funktionen mit Rückgabewerten landet die Rückgabe auf dem Programmstapel

④

Rekursive Funktionen sind die Funktionen, die sich selbst aufrufen.

unsigned $f(\text{unsigned } n)$

if ($n == 0$) {

return 1;

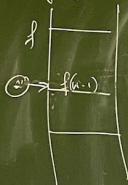
return $n + f(n-1)$;

}

int main {

$f(3)$;

}



Programmstapel

$f: n$	3	1. Inkarnation
$f: n$	2	
$f: n$	1	2. Inkarnation
$f: n$	0	
$f: n$	0	3. Inkarnation
$f: n$	0	
$f: n$	0	4. Inkarnation
$f: n$	0	

void endless_dive() {
double x ;
endless_dive();
}

⑤